

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Curso: CE-1103 Algoritmos y Estructuras de Datos I.

Trabajo extraclase 2 Benchmark y visualización de estructuras de datos

Profesor: MSc. Andrés Vargas

Integrantes:

Ezequiel Bonilla Vega: 2025098474

Manuel Antonio Rojas L: 2020072613

Luis Fernando Ortiz M: 2022206801

Brainer Chacón O: 2021039600

18 de mayo de 2026

I semestre

Resumen:

Este informe detalla el funcionamiento de la aplicación. Definiendo las operaciones correspondientes a la creación de estructuras, utilización de las mismas y la recolección de sus datos; configuración de parámetros R, W y N así como la creación de datos aleatorios según una semilla dada por el usuario. Adicionalmente, la representación gráfica de los árboles y el modo inspeccionar también se detallan en este informe.

Introducción:

Dentro de la carrera de Ingeniería en Computadores, en el curso Algoritmos y estructuras de Datos I; es importante que como futuro ingeniero conozcamos la manera en la que podemos modelar sistemas de datos, tanto para almacenarlos, buscarlos y eliminarlos.

Dichas estructuras poseen una complejidad de implementación tanto de espacio, memoria y dificultad a la hora de implementarla, existen diversos tipos que pueden funcionar para algunas tareas en específico pero no para otras, además de su complejidad a la hora de crearlas.

Esta aplicación busca comparar dichas estructuras en términos de tiempo (Cuanto duran ejecutando las diversas acciones) en busca de cuál cumple de manera más eficiente dicha función, además de brindar la posibilidad de ver una representación gráfica de los mismos con fines didácticos.

Objetivo:

Desarrollar una aplicación que compare diversas estructuras de datos vistas en clase, que visualice su duración en términos de tiempo, además de la posibilidad de graficarlos para una representación visual.

Marco teórico:

Dentro de la aplicación se utilizaron las siguientes estructuras: LinkedList, ArrayList, Binary Search Tree (BST), Red Black Tree (RBT), AVL Tree y Splay Tree; cada una de estas implementada a mano. La aplicación requiere un número de

repeticiones (R) la cuál indica cuantas veces se realizarán las acciones de: Inserción, Búsqueda y Eliminación.

En ocasiones, al iniciar una aplicación, al CPU se le dificulta ejecutar el código por primera vez, esto para mediciones puede incurrir en errores ya que teóricamente no se aprovecharía la potencia del equipo al ser las ejecuciones iniciales. Para evitar eso se agregó una nueva variable llamada W (Warm-up), la cuál ejecutará las acciones ya mencionadas la cantidad de veces indicada que no será contada para las mediciones, posterior a eso se ejecutarán las repeticiones (R).

Adicionalmente para probar la efectividad y funcionamiento de las estructuras los datos a insertar no son estáticos, esto quiere decir, no están definidos dentro del código. Los datos son generados de manera totalmente aleatoria, la cantidad a generar está a decisión del usuario. Para asegurar tener resultados fácilmente comprobables, estos datos aleatorios están ligados a una semilla, la cuál también puede insertar el usuario, la misma semilla de los mismos datos siempre.

Para medir los tiempos se utilizaron librerías y clases nativas de Java, obteniendo el tiempo de inicio y finalización pasado durante la acción y luego la diferencia para obtener el tiempo. El tiempo manejado es en milisegundos ya que las computadoras gracias a sus procesadores son bastante rápidas por lo que es difícil hacer este tipo de comparaciones.

Con respecto a la sección de “secuencia”, la idea principal se basó en volver a implementar las estructuras pero de una manera simplificada y adaptada exclusivamente a una demostración gráfica del proceso de inserción de datos, que en este caso se trata de solamente números enteros.

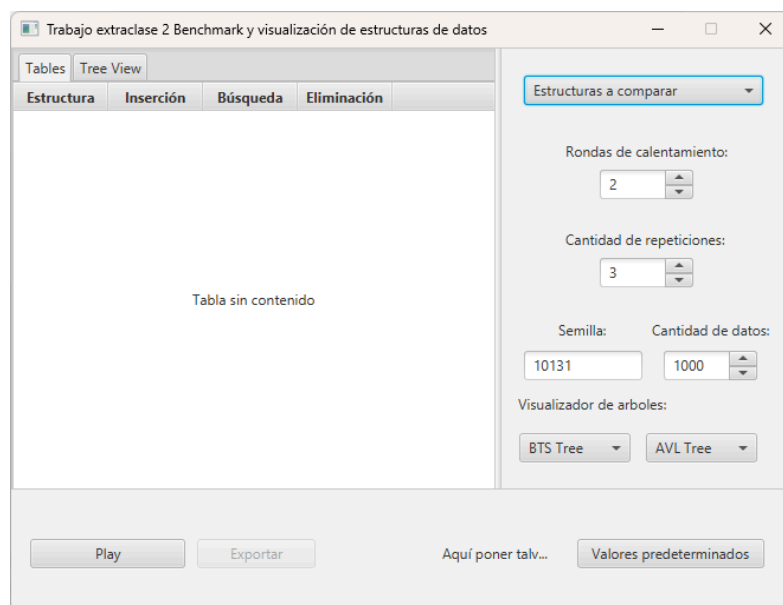
Para lograr la traslación lineal de los diferentes estados conforme cada árbol se comienza a poblar se implementó una lista doblemente enlazada que trabaja exclusivamente con una “interface” implementada por cada estructura especializada con el nombre de “Idrawable”. Esta contiene el contrato de “recorrerEjecutar” el cual permite a JavaFX encargarse de la lógica de dibujo de nodos o líneas por medio de un callback.

Esta lista doblemente enlazada junto con la mayor parte de la lógica gráfica vive dentro de una clase “controlador” con el nombre de “sequenceWindow”.

Otro aspecto importante de esta clase se trata de un método llamado “structureRouter”, este método se encarga de crear, poblar e inicializar el árbol que el usuario escoge por medio del “ChoiceBox”. En su núcleo se encuentra un case-switch que lee la opción en forma de “String” y a partir de este, escoge la inicialización adecuada.

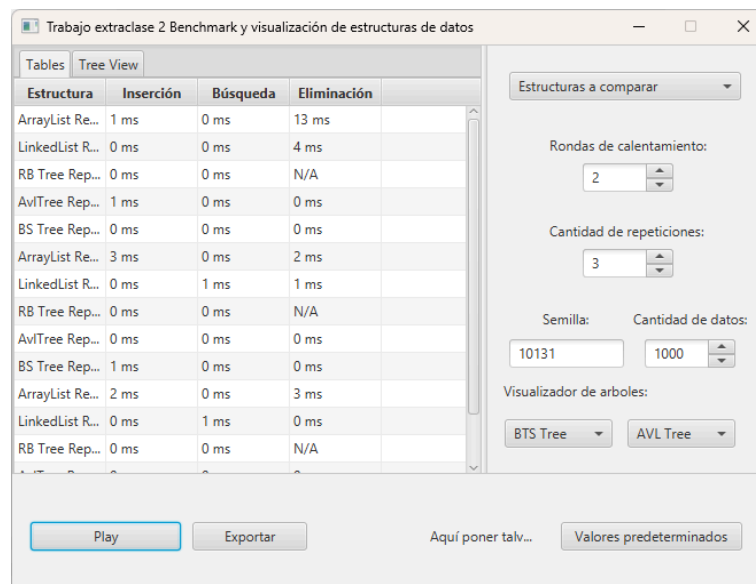
Por último, cabe mencionar el “text parser” implementado con la función de asistir al programa a la obtención de los datos ingresados por el usuario. Este lee un “TextField” que contiene la secuencia de datos separados por “,”. El parser logra separar cada dato por medio de la coma y consecuentemente se encarga de realizar un “cast” de “String” a “int” con el cual trabaja cada estructura especializada.

Descripción técnica de la aplicación:



La aplicación se compone de 3 ventanas principales, la primera sería la visualización de datos, la cuál al inicio se comporta como una tabla vacía pero al pulsar inicio se realizan las comparaciones y se llena la tabla; luego está el apartado de configuración en donde el usuario es capaz de modificar las estructuras a comparar, rondas de calentamiento (W), cantidad de repeticiones (R), la semilla que dicta los elementos aleatorios, la cantidad de datos y el visualizador de árboles correspondiente a su apartado propio; finalmente el último, donde se ubican los

botones de play (Correr comparación), valores predeterminados que corresponden a los datos precargados al iniciar la aplicación y un botón para exportar la tabla a un formato CSV una vez ejecutado el programa.



Dentro del programa, lee cuáles estructuras están seleccionadas, las crea y las coloca dentro de una lista. Posteriormente, dentro de un ciclo se llama la función capaz de hacer las comparaciones y agregar los datos a la tabla n veces, según la cantidad de R ingresada. Esto también sucede con W solamente que la aplicación para esas ejecuciones no las registra dentro de la tabla.

Al hacer esto, se puede obtener un resultado similar al de la imagen, es importante destacar que para lograr ejecutar el código de inserciones, búsquedas y eliminaciones, sin tener muchas comparaciones con if, se optó por utilizar una interfaz implementada para todas las estructuras que llamaran a sus respectivos métodos.

El botón exportar convierte las filas en datos para insertarlos en un nuevo archivo CSV, el cuál contiene la fecha de su creación dentro del nombre del archivo, y una representación en formato CSV de lo visto en la tabla en la aplicación, como en la siguiente imagen.

	A	B	C	D	E
1	Estructura	Inserción	Búsqueda	Eliminación	
2	LinkedList Repetición 0	0 ms	0 ms	4 ms	
3	RB Tree Repetición 0	0 ms	0 ms	N/A	
4	AvlTree Repetición 0	1 ms	0 ms	0 ms	
5	BS Tree Repetición 0	0 ms	0 ms	0 ms	
6	ArrayList Repetición 1	3 ms	0 ms	2 ms	
7	LinkedList Repetición 1	0 ms	1 ms	1 ms	
8	RB Tree Repetición 1	0 ms	0 ms	N/A	
9	AvlTree Repetición 1	0 ms	0 ms	0 ms	
10	BS Tree Repetición 1	1 ms	0 ms	0 ms	
11	ArrayList Repetición 2	2 ms	0 ms	3 ms	
12	LinkedList Repetición 2	0 ms	1 ms	0 ms	
13	RB Tree Repetición 2	0 ms	0 ms	N/A	
14	AvlTree Repetición 2	0 ms	0 ms	0 ms	
15	BS Tree Repetición 2	0 ms	0 ms	0 ms	
16					

Para ingresar al modo de secuencia solamente hace falta apretar el botón con su mismo nombre

Tables

Tree View

Estructura

Inserción

Búsqueda

Eliminación

No content in table

Estructuras a comparar

Rondas de calentamiento:
2

Cantidad de repeticiones:
3

Semilla: 10131 Cantidad de datos: 1000

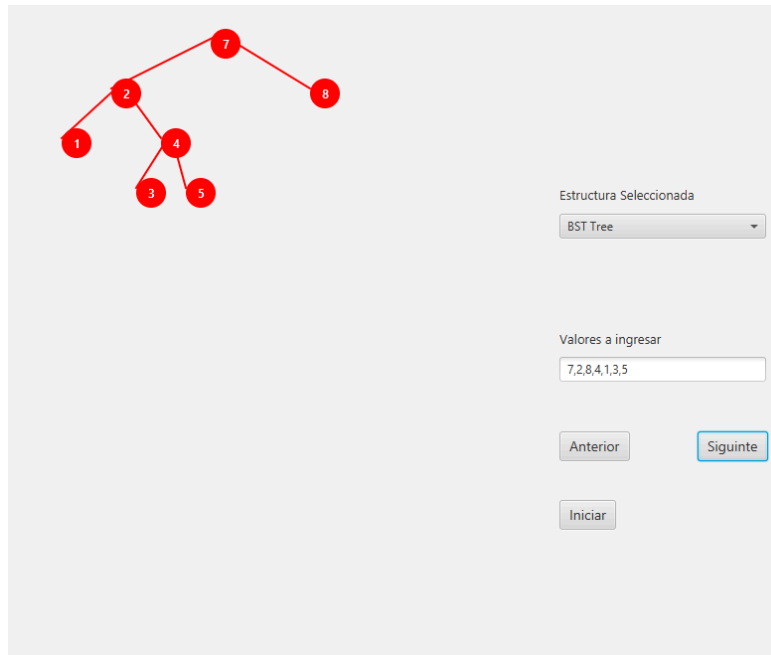
Visualizador de arboles:
BTS Tree AVL Tree

Play

Exportar

Modo Secuencia

Valores predeterminados



Se abre una nueva ventana con el modo seleccionado. Por medio del ChoiceBox se selecciona el tipo de árbol y se aprieta el botón de iniciar. El programa lee los datos que presenta el TextField y comienza con el proceso de graficación de cada inserción. Este proceso puede visualizarse paso a paso utilizando los botones de Anterior y Siguiente.

Conclusiones:

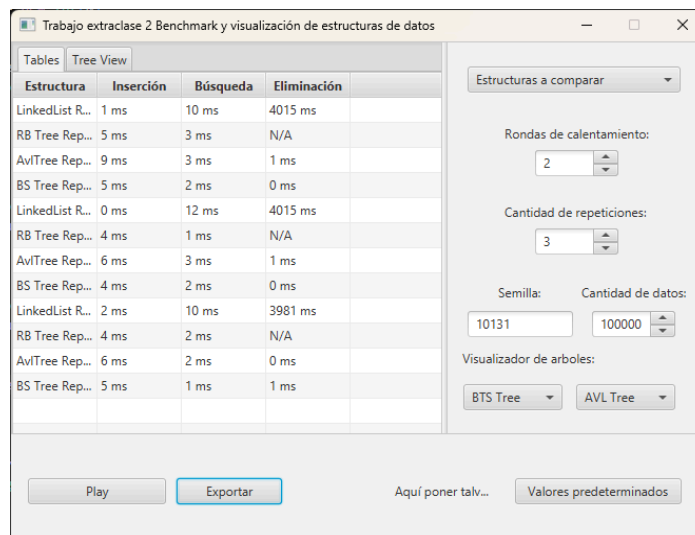
La estructura menos eficiente es el Array List, dada a la siguiente comparación W=2, R=3, N=10000, semilla=10131:

	A	B	C	D
1	Estructura	Inserción	Búsqueda	Eliminación
2	LinkedList Repetición 0	0 ms	0 ms	32 ms
3	RB Tree Repetición 0	1 ms	0 ms	N/A
4	AvlTree Repetición 0	0 ms	0 ms	0 ms
5	BS Tree Repetición 0	0 ms	0 ms	1 ms
6	ArrayList Repetición 1	162 ms	0 ms	153 ms
7	LinkedList Repetición 1	0 ms	1 ms	27 ms
8	RB Tree Repetición 1	0 ms	0 ms	N/A
9	AvlTree Repetición 1	0 ms	1 ms	0 ms
10	BS Tree Repetición 1	0 ms	0 ms	0 ms
11	ArrayList Repetición 2	148 ms	0 ms	159 ms
12	LinkedList Repetición 2	0 ms	0 ms	28 ms
13	RB Tree Repetición 2	1 ms	0 ms	N/A
14	AvlTree Repetición 2	1 ms	0 ms	0 ms
15	BS Tree Repetición 2	1 ms	0 ms	0 ms

Con 10000 datos vemos como las inserciones y eliminaciones son muchísimos más costosas con esta estructura, con un promedio de 103,3 ms en inserción y 114,6 ms en eliminación.

Este resultado es esperable al ser una estructura que para insertar y eliminar tiene que crear una nuevo arreglo correspondiente, aún así posee un buen promedio de tiempo en búsqueda.

Para la inserción de 100000 datos, se empezó a ralentizar la aplicación, al obtener el resultado, se concluyó lo siguiente:



La segunda peor estructura es el LinkedList, su inserción es rápida gracias a la referencia que existe dentro de sí misma del último dato, pero empieza a necesitar más tiempo para búsquedas con un promedio de 10,6 ms, y con un pésimo tiempo de eliminación promedio de 4003,6 ms, 40 segundos prácticamente.

Trabajo extracurricular 2 Benchmark y visualización de estructuras de datos

Estructura	Inserción	Búsqueda	Eliminación
RB Tree Rep...	44 ms	21 ms	N/A
AvlTree Rep...	71 ms	24 ms	13 ms
BS Tree Rep...	49 ms	18 ms	6 ms
RB Tree Rep...	45 ms	23 ms	N/A
AvlTree Rep...	72 ms	23 ms	14 ms
BS Tree Rep...	46 ms	18 ms	5 ms
RB Tree Rep...	44 ms	22 ms	N/A
AvlTree Rep...	71 ms	24 ms	14 ms
BS Tree Rep...	44 ms	18 ms	5 ms

Estructuras a comparar

Rondas de calentamiento: 2

Cantidad de repeticiones: 3

Semilla: 10131 Cantidad de datos: 100000

Visualizador de arboles: BTS Tree AVL Tree

Play Exportar Aquí poner talv... Valores predeterminados

Finalmente, con 1.000.000 de datos vemos una diferencia bastante grande entre los árboles, tardando más el AVL en inserciones y eliminaciones, pero con un tiempo promedio en búsqueda. Seguido estaría el BST y finalmente el RBT.

A pesar de usar términos como “Mejor o peor” realmente la implementación de las estructuras dependen totalmente del uso que se les vaya a dar, con este breve experimento podemos asumir que un ArrayList es muy rápido en búsquedas mientras no se inserten o eliminan muchos datos seguidamente, o que por ejemplo el Splay Tree es muy eficiente en búsquedas de un elemento muchas veces pero eso lo hace tardar más en inserciones y eliminaciones.

Bibliografía:

- Baeldung. (2024, 8 de enero). Guide to OpenCSV. Baeldung. Recuperado el 18 de mayo de 2026, de <https://www.baeldung.com/opencsvhttps://www.baeldung.com/opencsv>
- Google. (2026). Gemini (Versión 3 Flash) [Modelo fundacional de lenguaje grande]. <https://gemini.google.com>
- OpenFX (s.f). Java FX. <https://openjfx.io>
- Galles, D. (s.f.). Data structure visualizations. University of San Francisco. Recuperado el 18 de mayo de 2026, de <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>